

Introducción a Pandas aplicado a Ingeniería de Datos

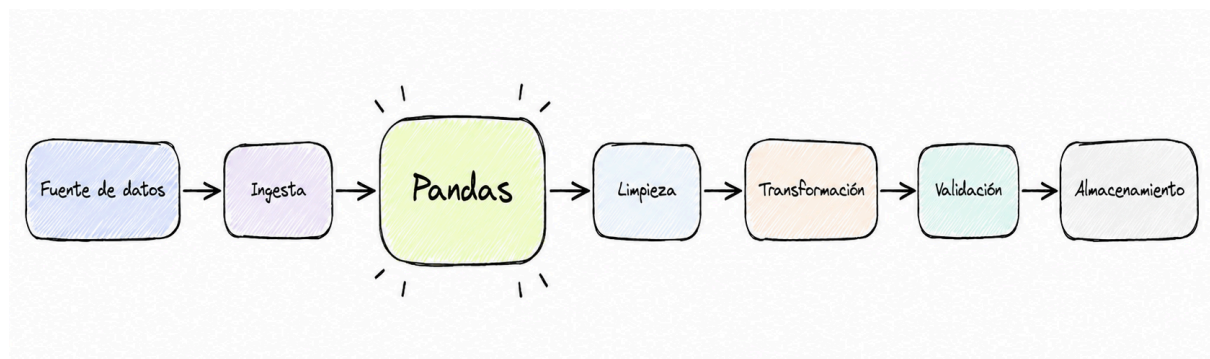
Introducción

En **ingeniería de datos**, una **gran parte del trabajo consiste en recibir datos, inspeccionarlos, limpiarlos, transformarlos y prepararlos**, para posteriormente ser utilizados en otros sistemas o incluso otras áreas.

En clases anteriores trabajamos con archivos utilizando JavaScript y Python, donde construimos pequeños **pipelines de datos de forma manual**. Sin embargo, cuando la cantidad de registros y columnas comienza a crecer, trabajar únicamente con listas, diccionarios y ciclos for, se vuelve insostenible.

Aquí es donde aparece **Pandas**, una biblioteca de Python, **escrita en C y CPython**, **especializada en manipulación y análisis de datos estructurados**. Permite trabajar **cómodamente** con información proveniente de archivos **CSV, JSON, EXCEL**, bases de datos y otras fuentes como **APIs**.

En un **pipeline** de datos, pandas puede intervenir principalmente en las etapas de:



Algo a tener en cuenta es que **Pandas no reemplaza a Python**, de hecho, es una herramienta construida sobre Python, que **nos proporciona estructuras especialmente diseñadas para trabajar con datos**.

¿Por qué Pandas?

Supongamos que recibamos información de ventas:

Python

```
ventas = [  
    { "producto": "Notebook", "precio": 850000, "cantidad": 2 },  
    { "producto": "Mouse", "precio": 15000, "cantidad": 5 },  
    { "producto": "Teclado", "precio": 35000, "cantidad": 3 }  
]
```

Si quisiéramos calcular por ejemplo, el importe total de cada venta, utilizando únicamente Python, tendríamos que hacer algo como esto:

Python

```
for venta in ventas:  
    venta["total"] = venta["precio"] * venta["cantidad"]
```

Que obviamente, no decimos que este mal, de hecho, va a funcionar perfectamente. Pero, imaginemos que estamos frente a un archivo con 2 millones de registros, con valores faltantes, registros duplicados, diferentes formatos de fechas. Ahora se complicó, ¿no? Podríamos intentar resolverlo con Python, y podríamos lograrlo, pero nuestro código crecerá tanto que llegará a un punto que será insostenible.

Con Pandas, podríamos hacer simplemente esto:

Python

```
df["total"] = df["precio"] * df["cantidad"]
```

Con solamente esta línea, estamos creando una columna llamada “total”, y estamos calculando el total de cada **fila** de nuestro **Dataframe**.

Series y DataFrames

Pandas introduce a python, dos nuevos tipos y esos son las **Series** y los **DataFrames**.

Una Serie es una estructura de datos unidimensional que almacena una secuencia de valores y un conjunto de etiquetas de índice asociadas a cada valor. Podemos imaginar a una serie como una sola columna de una tabla.

Un DataFrame en cambio, representa una estructura de datos, conformada por **filas y columnas**, de manera muy similar a por ejemplo: un tabla SQL o una hoja de cálculo de excel.

Instalación de Pandas

1. Creación de un entorno virtual: Antes de empezar a trabajar, es importante crear un entorno virtual de Python.

Shell

```
# para windows  
uv venv env && env\Scripts\activate
```

o

Shell

```
# para linux y macos  
uv venv env && env/bin/activate
```

2. Instalamos pandas utilizando uv y pip

Shell

```
uv pip install pandas
```

3. Importación dentro de un .py o un .ipynb

Python

```
import pandas as pd
```

Por convención, la comunidad de Python utiliza el alias “pd”, pero, sin el alias, podríamos utilizar directamente el nombre de la librería, pandas.

Python

```
import pandas
```

Y sería igual de funcional.

Estructuras fundamentales de Pandas

Como mencionamos más arriba, pandas posee dos estructuras principales:

- Series
- DataFrames

1. Series

Una series, representa una estructura unidimensional. La podemos crear con la c `pd.Series()`

Python

```
import pandas
```

```
precios = pd.Series([  
    1500,  
    2000,
```

```
3090,  
4000  
)  
print(precios)
```

Deberíamos obtener una salida como esta:

```
[2] ✓ 0 s ▶ import pandas  
  
precios = pd.Series([  
    1500,  
    2000,  
    3090,  
    4000  
)  
print(precios)
```

...	0	1500
	1	2000
	2	3090
	3	4000
	dtype: int64	

Pandas asigna un **índice** a cada elemento automáticamente.

2. DataFrames

El dataframe podemos verlo representado de la siguiente forma:

```
Python  
import pandas as pd  
  
datos = {  
    "producto": [  
        "Notebook",  
        "Mouse",  
        "Teclado"  
    ],
```

```
"precio": [
    850000,
    15000,
    35000
],
"cantidad": [
    2,
    5,
    3
]
}

df = pd.DataFrame(datos)
print(df)
```

Deberíamos obtener un resultado como este:

```
[3]
✓ 0s
datos = {
    "producto": [
        "Notebook",
        "Mouse",
        "Teclado"
    ],
    "precio": [
        850000,
        15000,
        35000
    ],
    "cantidad": [
        2,
        5,
        3
    ]
}

df = pd.DataFrame(datos)
print(df)
```

	producto	precio	cantidad
0	Notebook	850000	2
1	Mouse	15000	5
2	Teclado	35000	3

al igual que las series se identifican en índices, pero con la distinción que dentro de nuestro dataframe podemos trabajar con series(filas) y columnas.

RECORDAR:

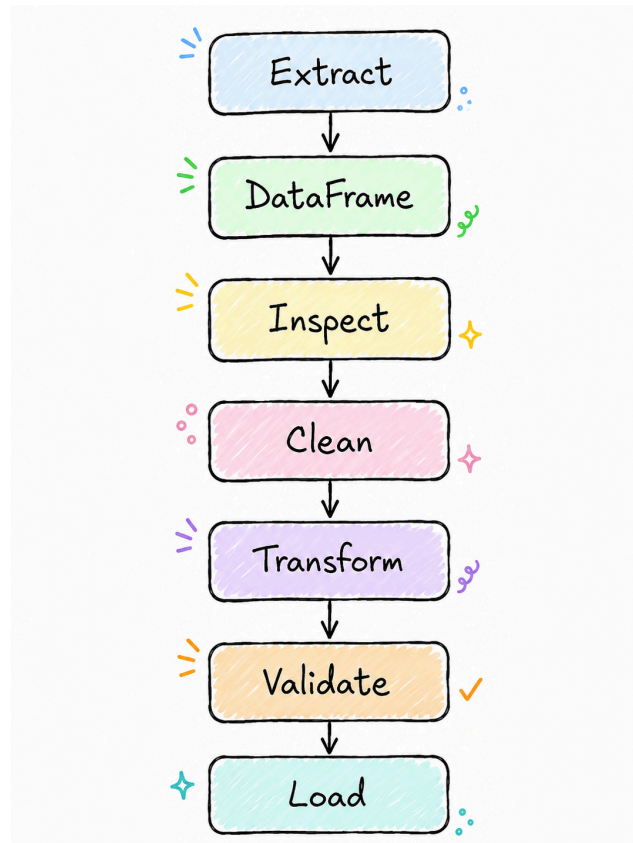
A la hora de querer o buscar crear en implementar nuevas columnas, es importante inferir en repetir la misma estructura, es posible que lo que suele fallar sean problemas en comas, uso de () o [].

Como podemos ver, pandas creó una columna por cada key de nuestro diccionario y agregó sus valores en cada columna correspondiente, y ahora estas conforman también filas.

entonces primero se crea la columna como key; y de ahí se crean los valores como filas.

Pandas dentro de un Pipeline de Datos

Podemos pensar en el trabajo de pandas mediante una secuencia:



Extract: Extraemos el contenido de un archivo, base de datos o incluso de una API.

DataFrame: Con pandas convertimos los datos en una estructura de filas y columnas.

Inspect: Realizamos un análisis para verificar los datos.

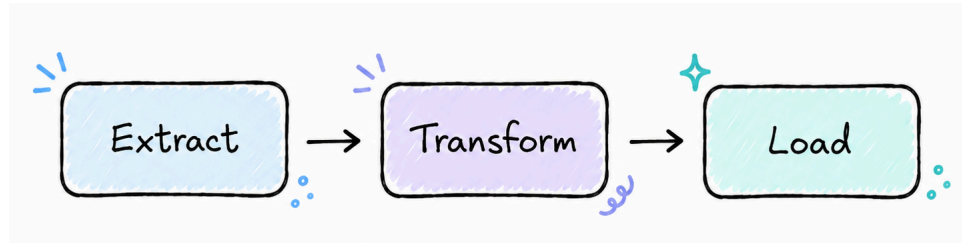
Clean: Limpiamos los valores duplicados, nulos o inválidos.

Transform: Transformamos los datos, aplicamos algunas estandarizaciones.

Validar: Validamos de que todo haya salido correctamente.

Load: Guardamos los datos limpios y los dejamos accesibles.

Este proceso se aproxima bastante al concepto de **ETL**.



Ingesta de Datos

Una de las **ventajas** de **Pandas** es que **puede cargar información desde diferentes fuentes**:

1. CSV: Archivos de texto separados por comas o por puntos y comas. En pandas tenemos un método llamado **read_csv()** y lo podemos utilizar de la siguiente forma:

Python

```
import pandas as pd
```

```
df = pd.read_csv("path del archivo")
```

2. JSON: Archivos separados por keys y values. En pandas, podemos utilizar un método llamado **read_json()**:

Python

```
import pandas as pd
```

```
df = pd.read_json("path del archivo")
```

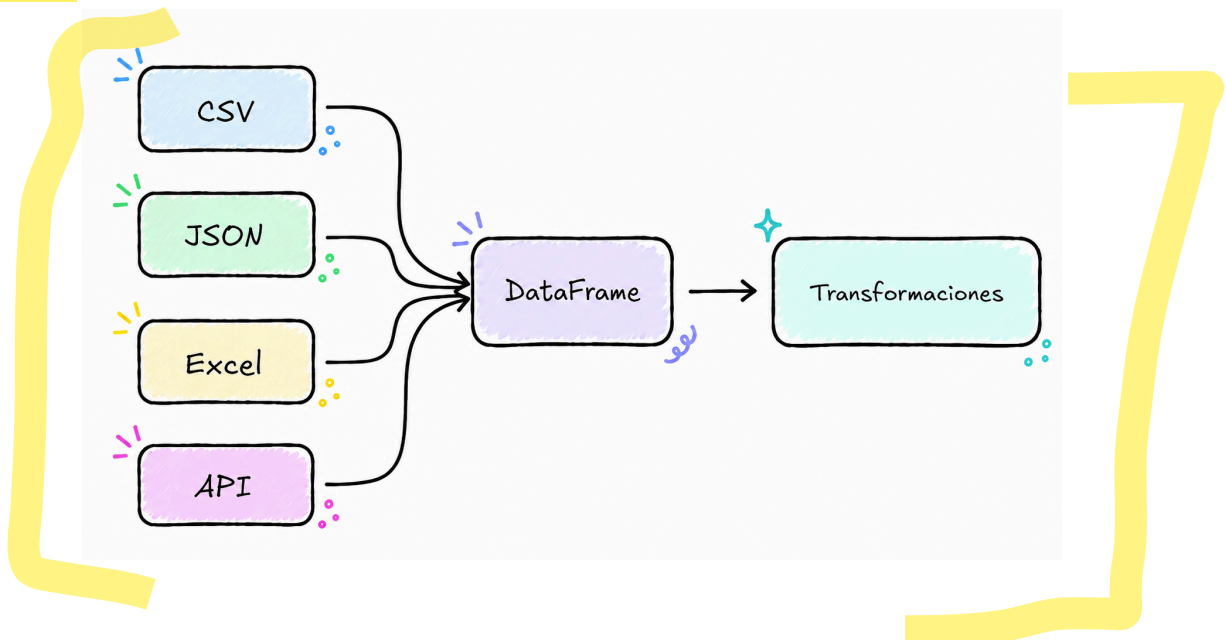

3. Excel: Normalmente las hojas de cálculo de excel, están en el siguiente formato **.xlsx**. En pandas tenemos el método **read_excel()**

Python

```
import pandas as pd
```

```
df = pd.read_excel("path del archivo")
```

Aunque la fuente de datos sea diferente, después de cargar los datos, normalmente terminaremos trabajando con un **DataFrame**. Esto nos permite desacoplar parcialmente nuestro procesamiento del formato original de los datos.



Procesamiento y preparación de datos con Pandas

Antes de modificar los datos debemos conocerlos, y es que uno de los errores más comunes es comenzar a transformar el dataset sin entender primero su estructura.

1. Acceder a las primeras filas: Lo podemos hacer fácilmente con el método **head()**

Python

```
df.head()
```

Por defecto nos mostrará las primeras 5 filas. Podemos pasarle como parámetro la cantidad de filas que queremos observar.

Python

```
df.head(10)
```

2. Acceder a las últimas filas: Podemos utilizar el método **tail()**, que similar a head, nos devuelve por defecto, las últimas 5 filas.

Python

```
df.tail()
```

Por defecto nos mostrará las últimas 5 filas. Podemos pasarle como parámetro la cantidad de filas que queremos observar.

Python

```
df.tail(10)
```

3. Dimensiones: Con el atributo **shape** podemos obtener las dimensiones de nuestro dataframe:

Python

```
df.shape
```

```
# Obtendremos una salida similar a: (1000, 8)
```

```
# que significa: 1000 filas y 8 columnas.
```

4. **Obtener las columnas:** podemos acceder a los nombres de las columnas con el atributo **columns**

Python

```
df.columns
```

Obtendrás una salida como esta:

```
Index([ 'id', 'producto', 'precio', 'cantidad', 'fecha' ],  
      dtype='object')
```

Esto resulta especialmente útil cuando recibimos un dataset que no conocemos.

5. **Tipos de datos:** con el atributo **dtypes** podemos obtener los tipos de datos correspondientes a cada columna:

Python

```
df.dtypes
```

Obtendrás una salida como esta:

Por ejemplo:

id	int64
producto	object
precio	float64
cantidad	int64
fecha	object

Conocer los tipos es importante porque los datos provenientes de archivos externos no siempre son interpretados correctamente.

Por ejemplo, una fecha puede ser interpretada simplemente como texto:

"2026-08-26"

6. Información General de un Dataframe: Podemos obtener información bastante importante de nuestro dataset, utilizando el método **info()**

Python

```
df.info()
```

Esta operación muestra:

- cantidad de registros;
- nombres de columnas;
- valores no nulos;
- tipos de datos;
- Uso aproximado de memoria.

7. Estadísticas rápidas: Para columnas numéricas, podemos usar:

Python

```
df.describe()
```

Obtendremos valores como:

- cantidad;
- promedio;
- desviación estándar;
- mínimo;
- máximo;
- percentiles.

8. Seleccionar columnas: Con pandas, podemos seleccionar directamente una sola columna:

Python

```
df["producto"]
```

Y como resultado, obtendremos una serie. También podemos elegir varias columnas al mismo tiempo

Python

```
df[[ "producto", "precio" ]]
```

Esto produce un nuevo DataFrame.

9. Eliminar columnas innecesarias: Los datasets reales muchas veces contienen información que no necesitamos.

Por ejemplo: Imaginemos que nuestro dataset contiene las siguientes columnas:

- id
- producto
- precio
- cantidad
- fecha
- comentario_interno
- debug
- campo_temporal

Podemos seleccionar solamente las columnas necesarias:

Python

```
df = df[[ "id", "producto", "precio", "cantidad", "fecha" ]]
```

Otra posibilidad es eliminar columnas:

Python

```
df = df.drop( columns=[ "comentario_interno", "debug",  
"campo_temporal" ] )
```

Desde la perspectiva de Ingeniería de Datos esto resulta importante para, por ejemplo:

- disminuir datos innecesarios
- simplificar pipelines
- reducir almacenamiento
- evitar transportar información que no será utilizada.

10. Filtrar Registros:

En pandas, podemos utilizar condiciones para filtrar el contenido de una columna:

Python

```
df["precio"] > 10000
```

Como resultado obtendremos una serie con valores booleanos, algo parecido a esto:

precio

True

False

False

True

Sin embargo, si lo combinamos con:

Python

```
df[df["precio"] > 10000]
```

Todos aquellos valores True, serán los que quedarán, el resto en False, serán descartados.

Podemos también combinar varias condiciones para realizar los filtros, por ejemplo:

Python

```
df[(df["precio"] > 100000) & (df["cantidad"] > 0)]
```

Donde:

& → AND

| → OR

~ → NOT

Muy parecido a lo que hacíamos al principio con JavaScript.

11. Valores faltantes: En datasets reales, es extremadamente común encontrarse con valores faltantes. En pandas son representados como NaN:

Ejemplo:

producto	precio
Notebook	850000
Mouse	NaN
Teclado	35000

Existe un método llamado **isna()** que nos permite detectarlos

Python

```
df.isna()
```

Sin embargo, con este método obtendremos un dataframe con valores booleanos.

Lo que se suele hacer es combinar con el método **.sum()**, donde obtendremos algo así como esto:

```
producto 0
precio 12
cantidad 3
```

Así podemos conocer rápidamente cuántos valores faltantes existen por columna.

12. Eliminar valores faltantes: Podemos eliminar los registros faltantes utilizando un método llamado **dropna()**. Sin embargo, no es recomendado utilizar indiscriminadamente, ya que puede provocar la pérdida innecesaria de información.

Python

```
df.dropna()
```

También podemos especificar columnas críticas:

Python

```
df = df.dropna(subset=[ "producto", "precio" ])
```

Ahora solamente serán eliminadas las filas donde **producto** o **precio** sean nulos.

13. Reemplazar valores faltantes: Existe un método en pandas que nos permite reemplazar los valores faltantes **fillna()**:

Python

```
df = df["cantidad"].fillna(0)
```

En este caso, estamos reemplazando todos aquellos valores **NaN** por el 0.

También podríamos combinarlo con otros métodos, por ejemplo:

Python

```
df["precio"] = df["precio"].fillna(df["precio"].mean())
```

o lo correcto sería:

Python

```
promedio = df["cantidad"].mean()  
  
df = df["cantidad"].fillna(promedio)
```

14. Registro de duplicados: Otra situación habitual es recibir registros duplicados. Pandas nos provee de un método bastante útil para esto **df.duplicated()**.

Python

```
df.duplicated()
```

Esto nuevamente nos devuelve un dataframe con valores booleanos. Pero podemos combinarlo con el método **sum()**.

Python

```
df.duplicated().sum()
```

También podemos eliminarlos con **df.drop_duplicates()**

Python

```
df = df.drop_duplicates()
```

También podemos considerar solamente una columna o columnas en específico:

Python

```
df = df.drop_duplicates(  
    subset=["id"]  
)
```

15. Normalizar texto

Uno de los problemas más frecuentes en pipelines de datos es encontrar categorías escritas de diferentes maneras. Con pandas, podemos normalizarlas de la siguiente forma:

Python

```
df["ciudad"] = (df["ciudad"].str.strip()).str.lower()
```

Con esto, quitamos los espacios y colocamos el contenido de esta columna a minúsculas.

16. Reemplazar valores: También podemos corregir categorías conocidas:

Python

```
df["ciudad"] = df["ciudad"].replace({
    "fsa": "formosa",
    "formoza": "formosa",
    "resis": "resistencia"
})
```

accede a la columna ciudad, aplica el método replace para reemplazar valores conocidos.

Esto puede formar parte de una etapa de **normalización** o de **transformación**.

17. Conversión de tipos: Pandas tiene varios métodos para hacer conversiones de tipos. Supongamos que queremos convertir los valores de una columna precio al tipo numérico:

Python

```
df["precio"] = pd.to_numeric(df["precio"])
```

Esto intentará convertir todos los valores al tipo numérico. Sin embargo, si encuentra un valor que no puede convertirse a numérico como por ejemplo un "asd", lanzará una excepción.

Podemos combinarla con el parámetro `errors="coerce"`, para que aquellos valores que no sea posible convertirlos a números, se conviertan directamente a NaN.

Python

```
df["precio"] = pd.to_numeric(df["precio"], errors="coerce")
```

También existe el método `.astype()`. A diferencia `to_numeric()`, este pide que como parámetro especifiquemos el tipo de dato a convertir, por ejemplo:

Python

```
df["precio"] = df["precio"].astype("int")
```

Algo a tener en cuenta es que, si no es posible convertir todos los valores de la columna, nos lanzará una excepción.

18. Conversión a fechas: Las fechas son particularmente importantes en la ingeniería de datos. Un dataset podría contener:

2026-08-21
2026/08/22
23-08-2026
fecha incorrecta

Podemos intentar convertir los valores con el método `to_datetime()`

Python

```
df["fecha"] = pd.to_datetime(df["fecha"], errors="coerce")
```

Aquellos valores que no sea posible convertir a datetime, pasarán a ser NaT

También podemos especificar el formato:

Python

```
df["fecha"] = pd.to_datetime(df["fecha"], format="%y -m% -%d", errors="coerce")
```

Con este formato, estamos especificando que vamos a convertir fechas como por ejemplo: 2026-06-01

Directivas de formato más comunes

- %Y: Año con 4 dígitos (ej. 2026).
- %m: Mes con dos dígitos (01 al 12).
- %d: Día del mes con dos dígitos (01 al 31).
- %H: Hora en formato de 24 horas (00 al 23).
- %M: Minutos (00 al 59).
- %S: Segundos (00 al 59).

19. Extraer información de fechas: Una vez que una columna ya posea el tipo de fecha adecuado, podemos extraer sus componentes:

Python

```
df["anio"] = df["fecha"].dt.year  
df["mes"] = df["fecha"].dt.month  
df["dia"] = df["fecha"].dt.day
```

Obtendremos un resultado como esto:

Python

```
fecha = 2026-08-26  
año = 2026  
mes = 8  
dia = 26
```

20. Crear columnas nuevas: Una de las operaciones más frecuentes en un pipeline es crear información derivada. Supongamos que tenemos una columna precio y una columna cantidad, basados en el resultado de la multiplicación de ambos, queremos obtener el total:

Python

```
df["total"] = ( df["precio"] * df["cantidad"] )
```

Obtendremos ahora:

producto	precio	cantidad	total
Mouse	15000	3	45000

21. Ordenar datos:

Podemos ordenar registros utilizando un método llamado `sort_values()`

Python

```
df.sort_values( by="precio" ) #orden ascendente
```

o

Python

```
df.sort_values( by="precio", ascending=False ) #orden descendente
```

También podemos ordenar utilizando múltiples columnas:

Python

```
df.sort_values(by=[ "ciudad", "fecha" ])
```

22. Agrupar información: Una operación extremadamente importante es `groupby()`:

Supongamos:

ciudad	total
Formosa	15000
Formosa	25000
Clorinda	10000
Clorinda	30000

Podemos calcular ventas totales por ciudad:

```
Python |
df.groupby("ciudad")["total"].sum()
```

Resultado:

Clorinda	40000
Formosa	40000

Combinar datasets: En ingeniería de datos frecuentemente necesitamos combinar información proveniente de distintas fuentes:

Por ejemplo:

clientes.csv

cliente_id	nombre
1	Ana
2	Carlos

ventas.csv

venta_id	cliente_id	total
101	1	5000
102	2	9000

Podemos combinarlos usando el método `.merge()`

Python

```
resultado = pd.merge( ventas, clientes, on="cliente_id" )
```

El resultado sería:

venta_id	cliente_id	total	nombre
101	1	5000	Ana
102	2	9000	Carlos

Esta operación es conceptualmente similar a un: JOIN de SQL.

23. Exportar resultados: Una vez aplicado las transformaciones y validaciones correspondientes, ya podemos almacenarla la información nuevamente:

CSV: con el método `to_csv()`

Python

```
df.to_csv("ventas_limpias.csv", index=False )
```

JSON: con el método `to_json()`

Python

```
df.to_json("ventas_limpias.json", index=False )
```

Excel: con el método `to_excel()`

Python

```
df.to_excel("ventas_limpias.xlsx", index=False )
```